division.hs.

2.3 INTERVALOS.

Los intervalos tambien conocidos como ranges o rangos se generan utilizando la sintaxis de aristas [inicio.. fin]. Esta es una forma muy conveniente y legible de crear listas de elementos secuenciales.

Intervalos básicos (Sintaxis elemental)
Para crear una lista secuencial, simplemente indicas el principio y el final separados por dos puntos.

- [1..10] genera [1,2,3,4,5,6,7,8,9,10]
- ['a'..'z'] genera el abecedario en minúsculas:
['a','b','c',...,'z']

Definir un paso.
Si no quieres que avance de uno en uno, puedes especificar el segundo elemento de la serie antes de los dos puntos. Haskell calculará la diferencia (el paso) automáticamente.

- [2,4..10] genera los números pares: [2,4,6,8,10] (el paso es +2)
- [10,9..1] genera una cuenta regresiva:
[10,9,8,7,6,5,4,3,2,1] (el paso es -1)

Intervalos infinitos
Debido a que Haskell utiliza evaluación perezosa puedes

RMAZH

Scanned with CamScanner

crear rangos que no tienen un límite final. El programa solo calculará los números que necesites en el momento. [1..] genera la lista infinita de números naturales [1,2,3,4,5,...]

[5,10..] genera los multiplos de 5 de forma infinita [5,10,15,20,...]

Para usarlos sin que el programa se quede colgado en un bucle infinito se suelen combinar con funciones como take.

Haskell
take 5 [1..].

2.4 Operadores.

Un operador es fondamentalmente una función que uno o dos argumentos Caridad 102). La diferencia principal frente a las funciones tradicionales con nombres de letras (como div o mod) radica en su sintaxis de uso y en las reglas estrictas que rigen su precedencia y asociatividad.

Funciones Infijas Prefijas.
Por defecto, los operadores en Haskell otilizan notación por infija.

Scanned with CamScanner

Haskell

```
resultado = 5+3 -- Notación infija estandar.
```

Sin embargo cualquier operador infijo puede convertirse en una función prefija (donde el nombre del operador va al principio y recibe ambos argumentos) si se encierra entre parentesis ():

```
resultado = (+) 5 3 -- equivale a 5+3
```

Por el contrario, cualquier función alfanúmerica normal de dos argumentos (como div) puede utilizarse como un operador infijo si se encierra entre acentos graves ``:

```
resultado = 10 `div` 2 -- Equivalente a div 10 2
```

Haskell permite aplicar parcialmente un operador infijo utilizando on solo operando, ya sea a la izquierda o la derecha, encerrando la expresión entre paréntesis.
Esto genera funciones altamente legibles

Sección derecha (Opx): Equivale a \y -> y Opx

Haskell

```
multiplicarPorDos = (* 2)
-- multiplicarPorDos 5 => 10.
```

Scanned with CamScanner

2.7 Evaluación perezosa

Es el comportamiento predeterminado (default). A diferencia de la mayoría de los lenguajes imperativos o funcionales estrictos (como java, Python o C++), Haskell no evalúa una expresión en el momento en que se define o se pasa como argumento, sino únicamente cuando su resultado es indispensable para producir el valor final que se va a mostrar o procesar.

Como funciona bajo el capó (Thunks)

Cuando defines una expresión en Haskell, el sistema no calcula el resultado inmediatamente; en su lugar, crea un objeto interno llamado thunk

Creación: El thunk se crea y se pasa por el programa sin resolverse.

Evaluación: Cuando una función externa (como print, (+) o un patron de coincidencia) exige conocer el valor concreto, el thunk se evalua.

Memorización: Una vez evaluado, el thunk es reemplazado por su valor calculado. Si se vuelve a invocar esa misma variable o expresión, nunca se vuelve a recalcular, se devuelve directamente el valor almacenado.

Scanned with CamScanner